MANIPAL UNIVERSITY
JAIPUR

Department of Computer Science and Engineering

Artificial Intelligence and Soft Computing (CSE3121)

Algorithmic Implementation of AI Approaches with Viva

Genetic Algorithm for the Travelling Salesman Problem

A study on 15 Indian cities, validated against the exact optimum

Submitted by: Shashwat Shikhar Dwivedi

Roll No.: 2427030605

Submitted to: Dr. Neha Chaudhary

Academic Year 2026–27

Abstract

This report presents an implementation of a Genetic Algorithm (GA) for the Travelling Salesman Problem (TSP) over 15 Indian cities. Tours are encoded as permutations and evolved using tournament selection, Order Crossover, inversion mutation and elitism. Because the instance is small enough, the exact optimum (7478.9 km) is computed with the Held–Karp dynamic programme, which gives a ground truth for judging the GA. Over 10 independent runs the GA reaches a mean tour length of 7526.9 km (0.6% above optimal) and finds the true optimum in 5 of 10 runs, whereas random search with the same evaluation budget is 32.8% above optimal and the nearest-neighbour heuristic is 22.6% above. Experiments on mutation rate and population size show how each parameter affects solution quality.

Contents

1 Introduction **2**

1.1 Problem statement 2

1.2 Why a Genetic Algorithm 2

1.3 Objectives 2

2 Methodology **2**

2.1 Problem data 2

2.2 GA design 2

2.3 Algorithm 4

2.4 Exact baseline: Held–Karp 4

2.5 Implementation 4

3 Results **5**

3.1 Single run 5

3.2 Comparison with baselines 6

3.3 Effect of mutation rate 6

3.4 Effect of population size 7

4 Complexity Analysis **7**

5 Limitations **8**

6 Conclusion **8**

1 Introduction

1.1 Problem statement

Given a set of n cities and the distance $d(i, j)$ between every pair, find the shortest closed route that visits each city exactly once and returns to the start. For a tour $\pi = (\pi_1, \ldots, \pi_n)$ the objective is

$$L(\pi) = \sum_{i=1}^{n} d\big(\pi_i, \pi_{(i \bmod n)+1}\big).$$

For $n = 15$ there are $(15-1)!/2 \approx 4.4 \times 10^{10}$ distinct tours, so exhaustive enumeration is impractical, and the problem is NP-hard in general.

1.2 Why a Genetic Algorithm

A GA is a population-based metaheuristic inspired by natural selection. Instead of improving one solution it maintains a population of candidates and repeatedly applies selection, crossover and mutation. The population explores several regions of the search space in parallel, and recombination lets good partial routes from different tours be combined.

1.3 Objectives

- Implement a GA for TSP from scratch in Python.
- Validate it against the exact optimum computed by Held–Karp.
- Compare it with random search and nearest-neighbour baselines.
- Study the effect of mutation rate and population size.

2 Methodology

2.1 Problem data

The 15 cities are Delhi, Mumbai, Kolkata, Chennai, Bengaluru, Hyderabad, Ahmedabad, Pune, Jaipur, Lucknow, Bhopal, Patna, Chandigarh, Guwahati and Kochi. Each is given by its latitude and longitude, and the distance between two cities is the great-circle (haversine) distance

$$d = 2R \arcsin \sqrt{\sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}}, \qquad R = 6371 \text{ km}.$$

All pairwise distances are precomputed into a 15×15 matrix.

2.2 GA design

Table 1 summarises the design, and the following subsections explain each choice.

Table 1: GA design and parameters.

Component	Choice
Chromosome	Permutation of city indices
Fitness	$1/L(\pi)$ (selection compares L directly)
Selection	Tournament, size $k = 5$
Crossover	Order Crossover (OX1), probability 0.9
Mutation	Segment inversion, probability 0.2
Elitism	Best 2 individuals copied unchanged
Population size	150
Generations	300

Encoding. A chromosome is a permutation of the integers $0, \ldots, 14$, so every chromosome is a valid tour. A binary encoding would produce many invalid tours with repeated or missing cities.

Fitness. Fitness is $1/L$, so shorter tours score higher. Tournament selection only needs an ordering, so the implementation compares tour lengths directly, which is equivalent.

Selection. Tournament selection draws $k = 5$ individuals at random and returns the shortest tour among them. The tournament size controls selection pressure: larger k favours the fittest more strongly but risks premature convergence.

Order Crossover (OX1). Ordinary one-point crossover would duplicate some cities and drop others. OX1 copies a random slice from parent 1 and fills the remaining positions with the missing cities in the order they occur in parent 2, which always yields a valid permutation. Example with cut positions 2–4:

Parent 1	$[1, 2, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6, 7, 8]$
Parent 2	$[3, 7, 5, 1, 6, 8, 2, 4]$
Slice kept from parent 1	$[3, 4, 5]$
Remaining genes in parent 2 order	$[7, 1, 6, 8, 2]$
Child	$[7, 1, \mathbf{3}, \mathbf{4}, \mathbf{5}, 6, 8, 2]$

Mutation. Inversion reverses a random sub-route. It changes only two edges of the tour, a small local move similar to a 2-opt step, and keeps the tour valid. For example, inverting positions 2–5 of $[1, 2, 3, 4, 5, 6, 7, 8]$ gives $[1, 2, 6, 5, 4, 3, 7, 8]$.

Elitism. The two best tours of each generation are copied unchanged into the next, so the best tour found never gets worse.

2.3 Algorithm

Algorithm 1 Genetic Algorithm for TSP

Initialise population P of N_p random permutations
for $g = 1$ **to** G **do**
  Evaluate tour length $L(\pi)$ for every $\pi \in P$
  Record the best tour found so far
  $P' \leftarrow$ the e best tours of P ▷ elitism
  while $|P'| < N_p$ **do**
    $p_1, p_2 \leftarrow$ two tournament selections from P
    with probability p_c: $c \leftarrow \mathrm{OX1}(p_1, p_2)$ **else** $c \leftarrow p_1$
    with probability p_m: $c \leftarrow \mathrm{Inversion}(c)$
    add c to P'
  end while
  $P \leftarrow P'$
end for
return best tour found

2.4 Exact baseline: Held–Karp

To measure the GA honestly, the exact optimum is computed with dynamic programming over subsets. Let $\mathrm{dp}[S][j]$ be the shortest path that starts at city 0, visits exactly the cities in S and ends at $j \in S$:

$$\mathrm{dp}[S \cup \{k\}][k] = \min_{j \in S} \big(\mathrm{dp}[S][j] + d(j,k)\big), \qquad L^* = \min_{j \neq 0} \big(\mathrm{dp}[\mathrm{all}][j] + d(j,0)\big).$$

This takes $O(n^2 2^n)$ time, which is feasible for $n = 15$ but not for large instances.

2.5 Implementation

The code is in the accompanying notebook `GA_TSP_Algorithmic_Implementation.ipynb` and uses only the Python standard library, NumPy, pandas and Matplotlib. The core operators are shown below.

```
def order_crossover(p1, p2):
    a, b = sorted(random.sample(range(N), 2))
    child = [None]*N
    child[a:b+1] = p1[a:b+1]
    fill = [g for g in p2 if g not in child[a:b+1]]
    pos = 0
    for i in range(N):
        if child[i] is None:
            child[i] = fill[pos]; pos += 1
    return child

def inversion_mutation(t):
    a, b = sorted(random.sample(range(N), 2))
    t = t[:]
    t[a:b+1] = reversed(t[a:b+1])
    return t
```

3 Results

3.1 Single run

Figure 1 shows the best tour from one run (seed 42) next to the exact optimal tour. This run finished at 7745.4 km, 3.56% above the optimum, so it settled in a local optimum. Repeated runs (Section 3.2) give the fuller picture.

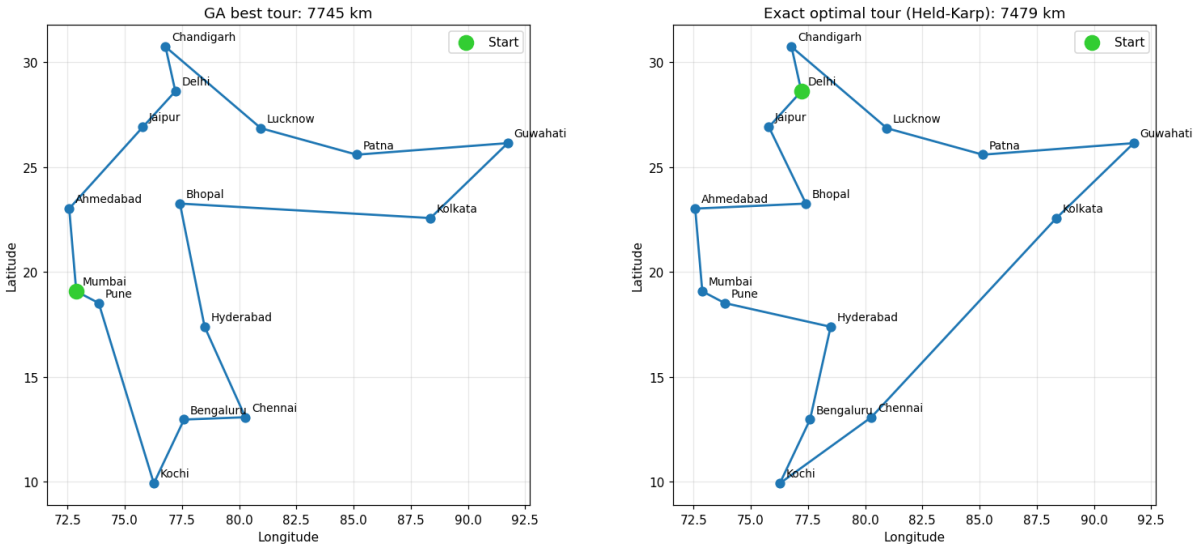


Figure 1: Left: best tour of a single GA run (seed 42, 7745 km). Right: exact optimal tour from Held–Karp (7479 km).

Figure 2 shows convergence. The population average drops quickly in the first generations as selection removes poor tours, while the best-so-far curve flattens once the population has converged.

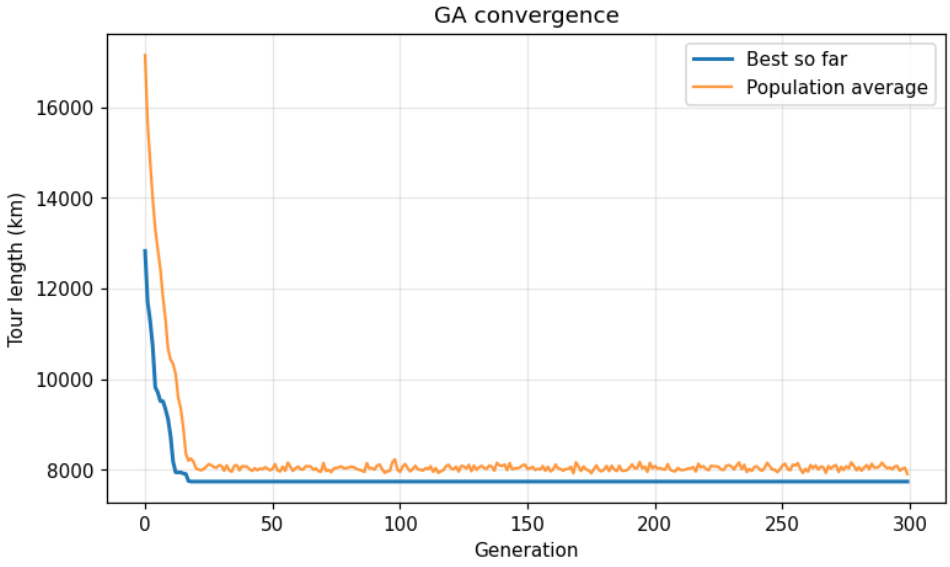


Figure 2: Convergence of the GA: best tour length so far and population average per generation.

3.2 Comparison with baselines

Table 2: GA versus baselines. Random search uses the same budget of $150 \times 300 = 45{,}000$ evaluations.

Method	Length (km)	Gap to optimum
Random search (mean of 3)	9930.4	32.8%
Nearest neighbour	9169.4	22.6%
Genetic Algorithm (mean of 10 runs)	7526.9	0.6%
Genetic Algorithm (best of 10 runs)	7478.9	0.0%
Exact optimum (Held–Karp)	7478.9	0.0%

The GA reached the exact optimum in 5 of 10 runs and was never far from it, while examining only 45,000 of roughly 4.4×10^{10} possible tours.

3.3 Effect of mutation rate

Each setting was run 8 times for 200 generations.

Table 3: Final tour length by mutation rate (8 runs each).

Mutation rate	Mean length (km)	Best length (km)
0.00	8079.0	7503.7
0.05	7660.8	7478.9
0.20	7532.2	7478.9
0.50	7545.6	7478.9
0.90	7478.9	7478.9

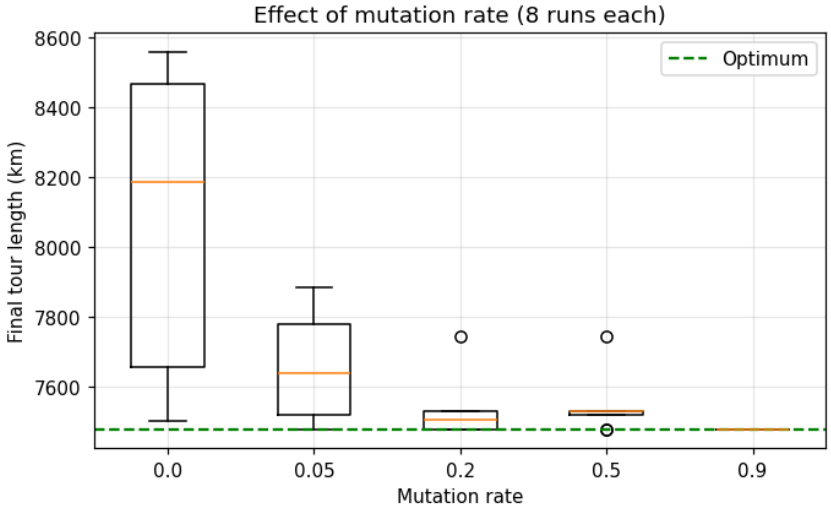


Figure 3: Spread of final tour lengths by mutation rate. The dashed line is the exact optimum.

With no mutation the GA performs worst: crossover only recombines genes already in the population, so diversity collapses and the search stalls. In this experiment higher mutation

rates did better, and the highest rate tested (0.9) found the optimum in every run. Inversion is a gentle move, so it tolerates high rates; a more disruptive operator would not.

3.4 Effect of population size

Every setting used the same budget of 45,000 evaluations, so smaller populations ran more generations.

Table 4: Final tour length by population size at equal evaluation budget (8 runs each).

Population	Generations	Mean length (km)	Best length (km)
10	4500	7683.1	7532.3
30	1500	7676.5	7478.9
100	450	7598.8	7478.9
300	150	7518.9	7478.9

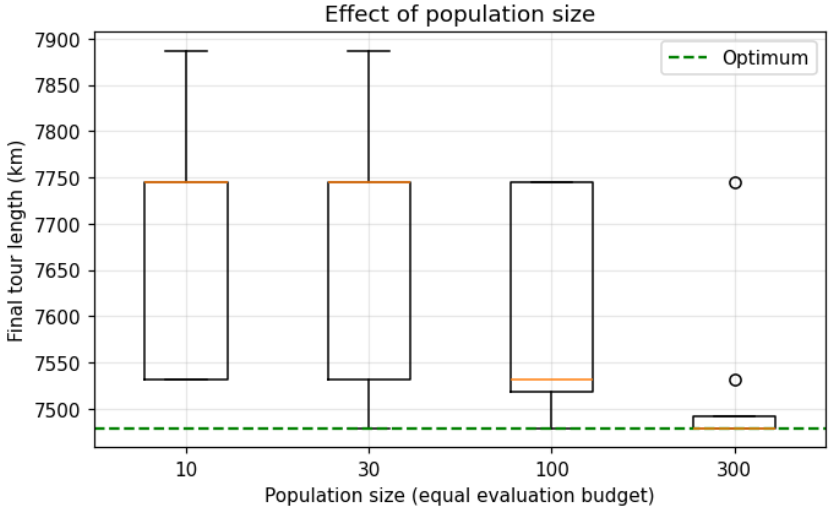


Figure 4: Spread of final tour lengths by population size at equal evaluation budget.

Larger populations did better. A population of 10 loses diversity quickly, so running 4500 generations does not help once it has converged, whereas 300 individuals keep more distinct partial routes alive for recombination.

4 Complexity Analysis

Let P be the population size, G the number of generations and n the number of cities.

- **GA time:** each generation evaluates P tours at $O(n)$ each, and crossover and mutation are $O(n)$ per child, giving $O(G \cdot P \cdot n)$.
- **GA space:** $O(P \cdot n)$ for the population.
- **Held–Karp:** $O(n^2 2^n)$ time and $O(n\, 2^n)$ space.

The GA scales polynomially in n, whereas the exact method grows exponentially and is impractical beyond roughly 25 cities.

5 Limitations

- The GA gives no optimality guarantee; a single run can stall in a local optimum, as the seed-42 run did.
- Results depend on the parameters and the random seed, so they should be reported over repeated runs.
- Premature convergence occurs when population diversity is lost, especially with small populations or no mutation.
- The experiments use one 15-city instance with 8–10 runs per setting, so the parameter trends are indicative rather than conclusive.

6 Conclusion

The GA found tours within 0.6% of the exact optimum on average for a 15-city TSP and reached the optimum in half of the runs, clearly outperforming random search and nearest neighbour at the same budget. The experiments show that mutation is necessary to avoid stagnation and that larger populations make better use of a fixed evaluation budget. Validating against Held–Karp turned a vague claim of a "good solution" into a measured optimality gap.